

Scenario Based Assignment

1. Scenario: A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyse the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

SOLUTION:

i. Greedy Best-First Search (GBFS):

GBFS selects the path with the lowest heuristic value (straight-line distance to the destination). It is very fast, making it useful in emergencies. However, it may choose risky or blocked routes because it ignores the actual travel cost, so it does not always find the best path.

ii. A Search:

A* uses both the actual travel cost $g(n)$ and the heuristic estimate $h(n)$, where $f(n) = g(n) + h(n)$. It balances speed and cost, avoiding expensive or risky routes while still moving toward the goal. This makes it more reliable in dynamic environments.

iii. Impact of Heuristic Accuracy:

- Accurate heuristic: Both GBFS and A* perform faster and find better routes.
- Inaccurate heuristic: GBFS may choose poor or blocked paths, while A* is less affected because it also considers the actual path cost.

iv. Effect of Dynamic Changes (Blocked Paths):

When roads become blocked, both algorithms may need to recalculate the route. GBFS can waste time exploring dead ends, whereas A* adapts better by considering updated costs, though it requires more computation.

v. Recommended Algorithm:

A* is the best choice because it provides an optimal and safer route by considering both travel cost and heuristic distance. Although it is slightly slower than GBFS, it is more suitable for emergency medicine delivery where safety, reliability, and minimum delay are critical.

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However, the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks: Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

SOLUTION:

Optimization Problem:

The AI system aims to minimize:

- Total waiting time
- Fuel consumption
- Traffic congestion

while continuously adjusting traffic signal timings based on changing traffic flow.

Why Traditional Search is Inefficient:

Traditional search methods are too slow because there are hundreds of intersections, creating a huge search space. They also cannot adapt quickly to dynamic traffic changes.

i. Hill Climbing

Hill Climbing starts with a signal timing plan and repeatedly makes small changes that improve traffic flow. It stops when no better neighbouring solution is found.

Limitations:

- May get stuck at a local optimum.
- Cannot explore better solutions beyond nearby states.

ii. Real-World Issues

- Local Maxima: One area has smooth traffic, but the overall city traffic is still poor.
- Plateaus: Many signal changes produce the same result, so the algorithm cannot decide where to move.
- Ridges: The best solution requires multiple coordinated signal changes, but Hill Climbing changes only one step at a time.

iii. Simulated Annealing / Genetic Algorithms

- Simulated Annealing: Sometimes accepts worse solutions temporarily, helping escape local optima.
- Genetic Algorithms: Combine and mutate multiple traffic signal plans to find better global solutions and adapt to changing traffic.

iv. Best Approach

Genetic Algorithm is the best choice because it:

- Handles very large search spaces efficiently.
- Finds near-optimal global solutions.
- Adapts well to changing traffic conditions.
- Scales effectively for hundreds of intersections.

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- Explain why this problem requires an online search agent instead of offline search
- Analyse the challenges of partial observability and dynamic environments
- Describe how the rover builds and updates its internal model
- Compare online search with uninformed and informed search strategies
- Justify the most suitable approach considering uncertainty, adaptability, and safety

SOLUTION:

I. Why Online Search Instead of Offline Search

The rover does not have a complete map of Mars. It must explore, gather information, and make decisions while moving. Therefore, **online search** is required because it plans and acts at the same time.

ii. Challenges of Partial Observability and Dynamic Environments

- **Partial Observability:** The rover can only sense nearby terrain, so it has incomplete information.
- **Dynamic Environment:** New obstacles, loose rocks, or changing terrain may appear, requiring continuous route updates.

iii. Internal Model

The rover:

- Collects sensor data about the surroundings.
- Updates its internal map as it explores.
- Marks safe paths and hazardous areas.
- Uses the updated model to make better navigation decisions.

iv. Online Search vs. Uninformed and Informed Search

Online Search	Uninformed Search	Informed Search
Explores while moving	Needs complete map	Uses heuristics but still needs map
Works with unknown environments	Not suitable for unknown terrain	Less effective without full information
Continuously updates decisions	Fixed search path	Depends on available knowledge

v. Best Approach

Online Search Agent is the most suitable approach because it:

- Handles uncertainty effectively.
- Adapts to new obstacles in real time.
- Works with limited sensing capability.
- Ensures safer navigation and efficient sample collection.

4. Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks:

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyse real-world challenges and justify the best strategy for scalability

SOLUTION:

Constraint Satisfaction Problem (CSP)

The exam timetable is a CSP where exams must be assigned to time slots and halls while satisfying all constraints.

i. Variables, Domains, and Constraints

- Variables: Exams (e.g., Math, Physics, Chemistry).
- Domains: Available time slots and exam halls.
- Constraints:
 - No student has overlapping exams.
 - Limited exam halls must not be exceeded.
 - Some subjects must be scheduled before others.
 - Faculty must be available.
 - Special accommodation requirements must be met.

ii. Backtracking Search

Backtracking assigns a time slot and hall to each exam. If a constraint is violated, it goes back (backtracks) and tries a different assignment until a valid timetable is found.

iii. Constraint Propagation (Forward Checking)

Forward checking removes invalid time slots from the remaining exams after each assignment. This reduces unnecessary searches and detects conflicts early, making the process faster.

iv. Real-World Challenges and Best Strategy

Challenges:

- Large number of courses and students.
- Many constraints to satisfy simultaneously.
- Last-minute changes in faculty or hall availability.

Best Strategy:

Use Backtracking with Forward Checking (Constraint Propagation) because it:

- Reduces the search space.
- Detects conflicts early.
- Produces valid timetables efficiently.
- Scales better for large universities with many courses and students.

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, make decisions within strict time limits, Evaluate complex game states with multiple possible moves, the decision tree is extremely large, making computation expensive.

Tasks:

- Explain how the Minimax algorithm models this problem
- Analyse the computational challenges of large game trees
- Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- Design an evaluation function and justify the factors considered
- Recommend a strategy for balancing optimality and real-time performance

SOLUTION:

i. Minimax Algorithm

The **Minimax** algorithm models the game as a two-player competition. It chooses the move that **maximizes the AI's chance of winning** while assuming the opponent always makes the best possible move.

ii. Computational Challenges

- The game has a **very large decision tree**.
- Many possible moves at each step increase computation time.
- Real-time decisions must be made quickly, making full tree search impractical.

iii. Alpha-Beta Pruning

Alpha-Beta Pruning improves Minimax by eliminating branches that cannot affect the final decision.

- **Alpha (α):** Best value found for the AI (maximiser).
- **Beta (β):** Best value found for the opponent (minimizer).
- When $\alpha \geq \beta$, the remaining branches are skipped because they cannot produce a better result.

- This reduces the number of nodes evaluated, making the AI much faster while still producing the same optimal move.

iv. Evaluation Function

The evaluation function scores each game state based on:

- Number and strength of units.
- Resources available.
- Territory or map control.
- Health of important structures.
- Distance to achieving victory.

These factors help the AI estimate how strong its current position is.

v. Best Strategy

Use **Minimax with Alpha-Beta Pruning** and a good evaluation function because it:

- Reduces computation time.
- Maintains near-optimal decisions.
- Handles large game trees efficiently.
- Meets real-time performance requirements.